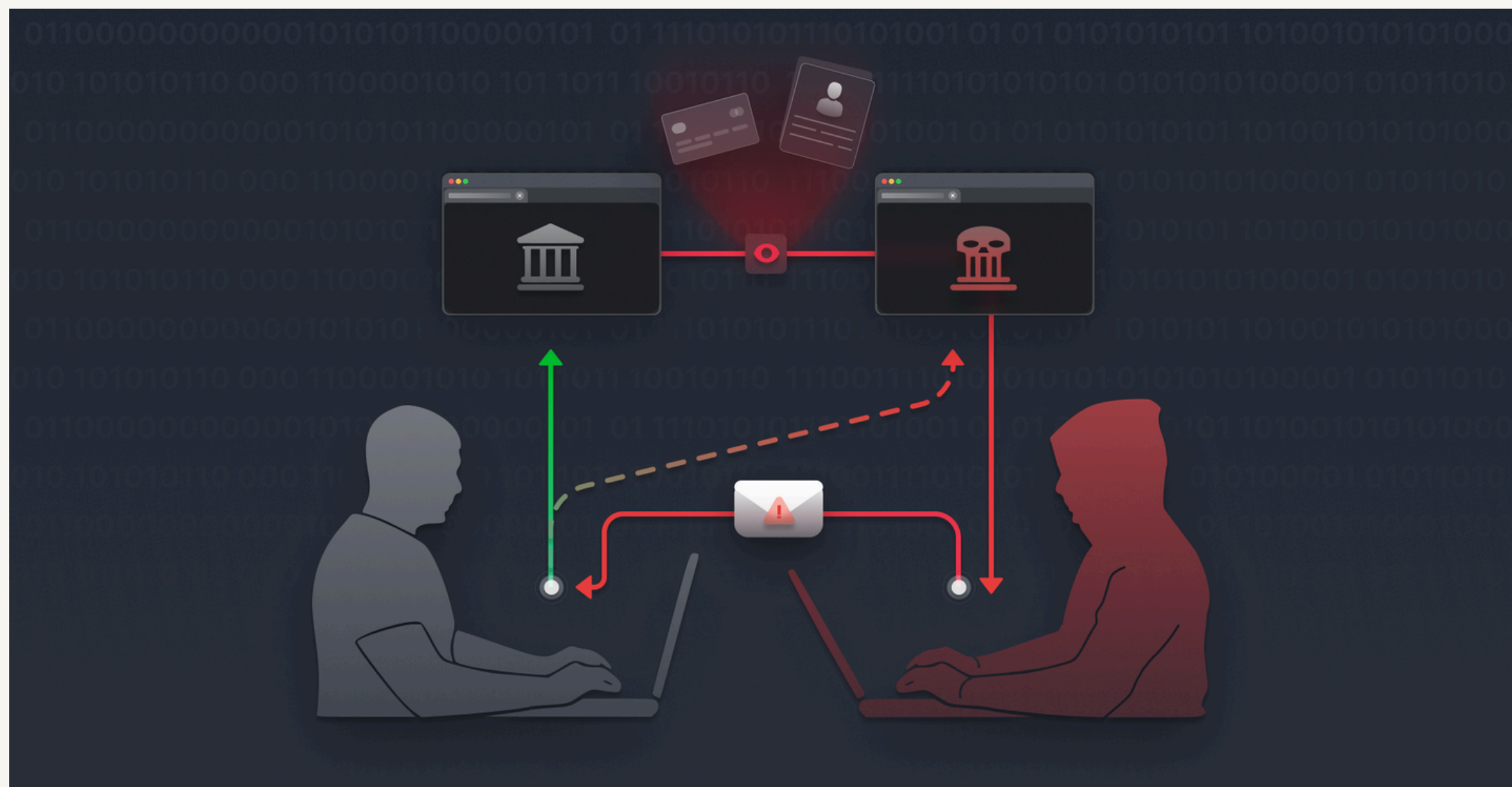
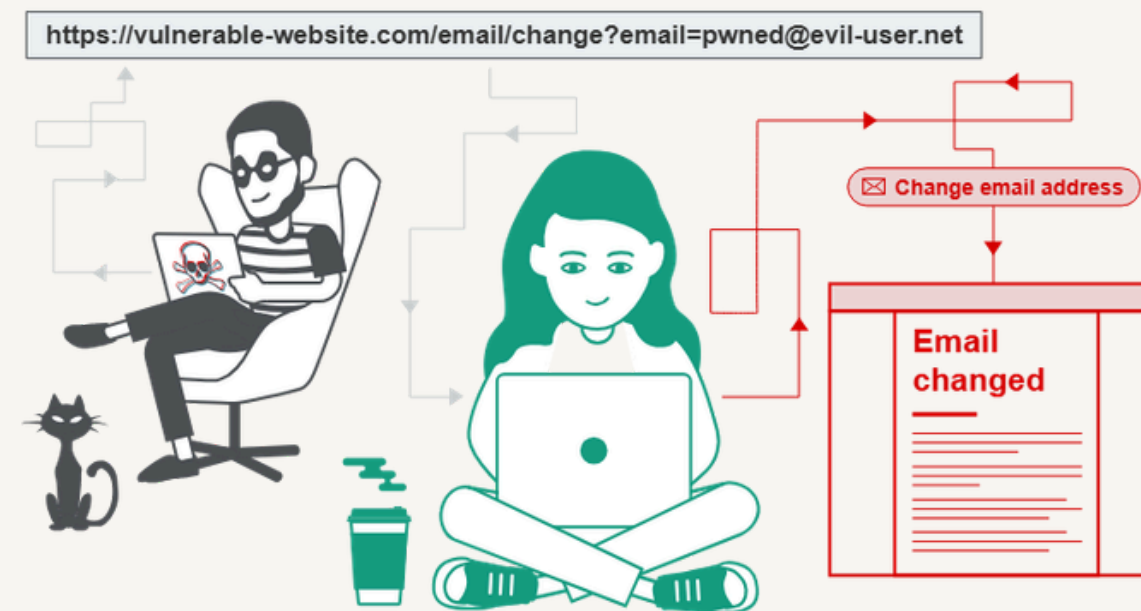


CSRF

(Cross-Site Request Forgery или
межсайтовая подделка запроса)



Подготовил: K i r o v K i r i l I A 2 4 0 4
Проверила: l e c t . u n i v . V i ş n e v s c h i V l a d a



КАК ЭТО РАБОТАЕТ?



КАК РАБОТАЕТ CSRF (МЕЖСАЙТОВАЯ ПОДДЕЛКА ЗАПРОСА)



```
<form action="https://bank.com/transfer" method="POST">
  <input name="to" value="attacker">
  <input name="amount" value="1000">
</form>

<script>
  document.forms[0].submit();
</script>
```

Generate Mermaid Diagram | Open Chat @mermaid-chart

```

form_csrf.php
1  <?php
2
3  session_start();
4
5  if (empty($_SESSION['csrf_token'])) {
6      $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
7  }
8  ?>
9
10 <form action="process.php" method="POST">
11     <input type="hidden"
12         name="csrf_token"
13         value="<e> htmlspecialchars($_SESSION['csrf_token']) ?>">
14     <input type="text" name="username">
15     <button type="submit">Отправить</button>
16 </form>
16
workspace/project/form_csrf.php

```

```

create.blade.php
1  <form action="{{ route('users.store') }}" method="POST">
2      @csrf
3
4      <input type="text" name="name">
5
6      <button type="submit">Создать</button>
7  </form>
7
workspace/project/resources/views/users/create.blade.php

```



```

process.php
1  <?php
2  session_start();
3  if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
4      http_response_code(405);
5      exit('Method Not Allowed');
6  }
7  $token = $_POST['csrf_token'] ?? '';
8  if (
9      empty($_SESSION['csrf_token']) ||
10     !hash_equals($_SESSION['csrf_token'], $token)
11 ) {
12     http_response_code(403);
13     exit('CSRF validation failed');
14 }
15 // Если токен правильный — обрабатываем запрос
16 $username = $_POST['username'] ?? '';
17 echo "Привет, " . htmlspecialchars($username);
17
workspace/project/process.php

```

"ЗАСТАВЬ СЕРВЕР СХОДИТЬ ТУДА, КУДА МНЕ НЕЛЬЗЯ".

Например, если сайт позволяет указать:
`/upload?url=https://example.com/image.jpg`
хакер может попробовать:
`/upload?url=http://localhost/admin`
И сервер пойдёт туда.

Что он может получить:

- 🔍 Узнать информацию о внутренних сервисах.
- 🔑 Получить данные, которые доступны только серверу.
- 🏠 Обратиться к **localhost** или внутренней сети, куда нет доступа из интернета.
- ☁ В некоторых облачных конфигурациях попытаться получить **служебные данные** о сервере.

1

SSRF VS CSRF

They sound similar, but they are very different!

Save for later

Java Developer Learning

2

What is CSRF?

Cross-Site Request Forgery

Tricks a logged-in user's browser into sending unwanted requests to a trusted website.

Attacker

User is logged in to bank.com

bank.com

The website thinks the request is from the user, not the attacker.

Prevention

- CSRF Tokens
- SameSite Cookies
- Check Origin / Referer headers
- Re-authentication for sensitive actions

Java Developer Learning

3

What is SSRF?

Server-Side Request Forgery

Tricks your server into making HTTP requests on the attacker's behalf.

Attacker

Your Server

Attacker may access sensitive data or internal services that are not exposed to the internet.

Prevention

- Allowlist URLs (trusted domains only)
- Block private IPs (127.0.0.1, 10.x.x.x, etc.)
- Block metadata endpoints
- Disable redirects
- Network level egress controls

Java Developer Learning

4

CSRF vs SSRF

Aspect	CSRF	SSRF
Full Form	Cross-Site Request Forgery	Server-Side Request Forgery
Victim	User's Browser	Your Server
Uses	User's authenticated session (cookies)	Server's network access
Exploits	Browser trust	Server trust
Goal	Perform unauthorized actions as the user	Access internal resources / services
Example	Transfer money, change email, update profile	Access admin API, internal DB, cloud metadata
Impact	Affects the user's account	Affects the server and internal systems

Java Developer Learning

5

Example

CSRF Example

```

```

If the user is logged in, the browser may send the request with cookies.

SSRF Example

```
{  "url": "http://169.254.169.254/latest/meta-data/"}
```

If the server fetches this URL, attacker can access cloud metadata / internal resources.

Remember

If the attack happens via the browser → Think **CSRF**
If the attack happens via the server → Think **SSRF**

Java Developer Learning

6

Interview Tip

Q: Why is CSRF generally not a concern for APIs secured with Bearer tokens in the Authorization header?

A: Because browsers do not automatically attach Bearer tokens to cross-site requests. Unlike cookies, the token must be explicitly added by client-side code, making classic CSRF attacks much harder. If the token is stored in cookies instead, CSRF protection may still be necessary depending on the application's design.

Real-World Analogy

CSRF: Someone tricks your trusted assistant into signing a document because the assistant recognizes your signature and assumes the request came from you.

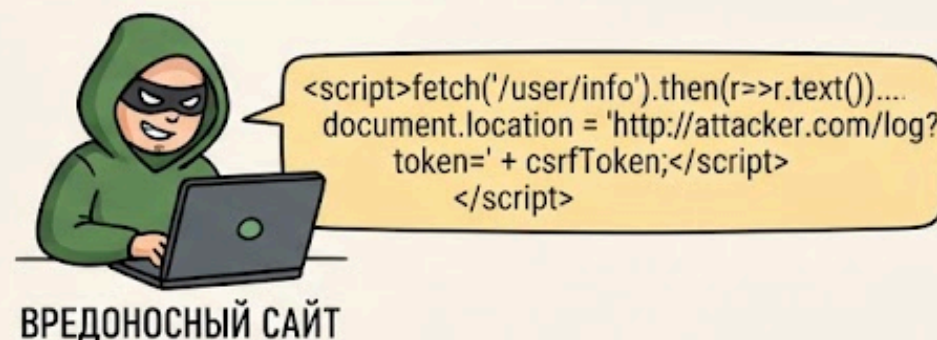
SSRF: Someone tricks your office receptionist into making a call to a restricted internal number that outsiders cannot dial directly.

Java Developer Learning

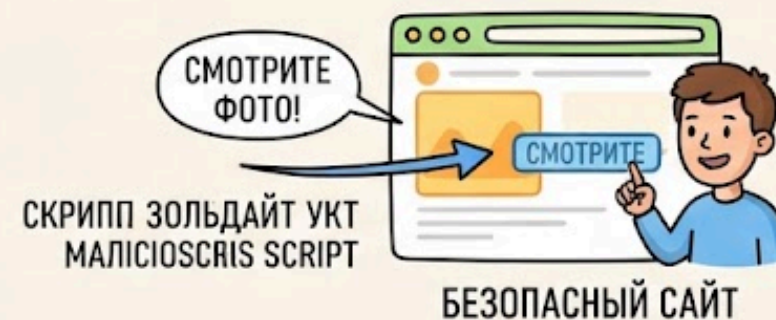
УЯЗВИМОСТЬ ЧЕРЕЗ XSS

КАК XSS ИСПОЛБЗУЕТСЯ ДЛЯ ПОЛУЧЕНИЯ CSRF ТОКЕНА

- 1 ЗЛОУМЫШЛЕННИК ВНЕДРЯЕТ
ВРЕДНОСНЫЙ СКРИПТ



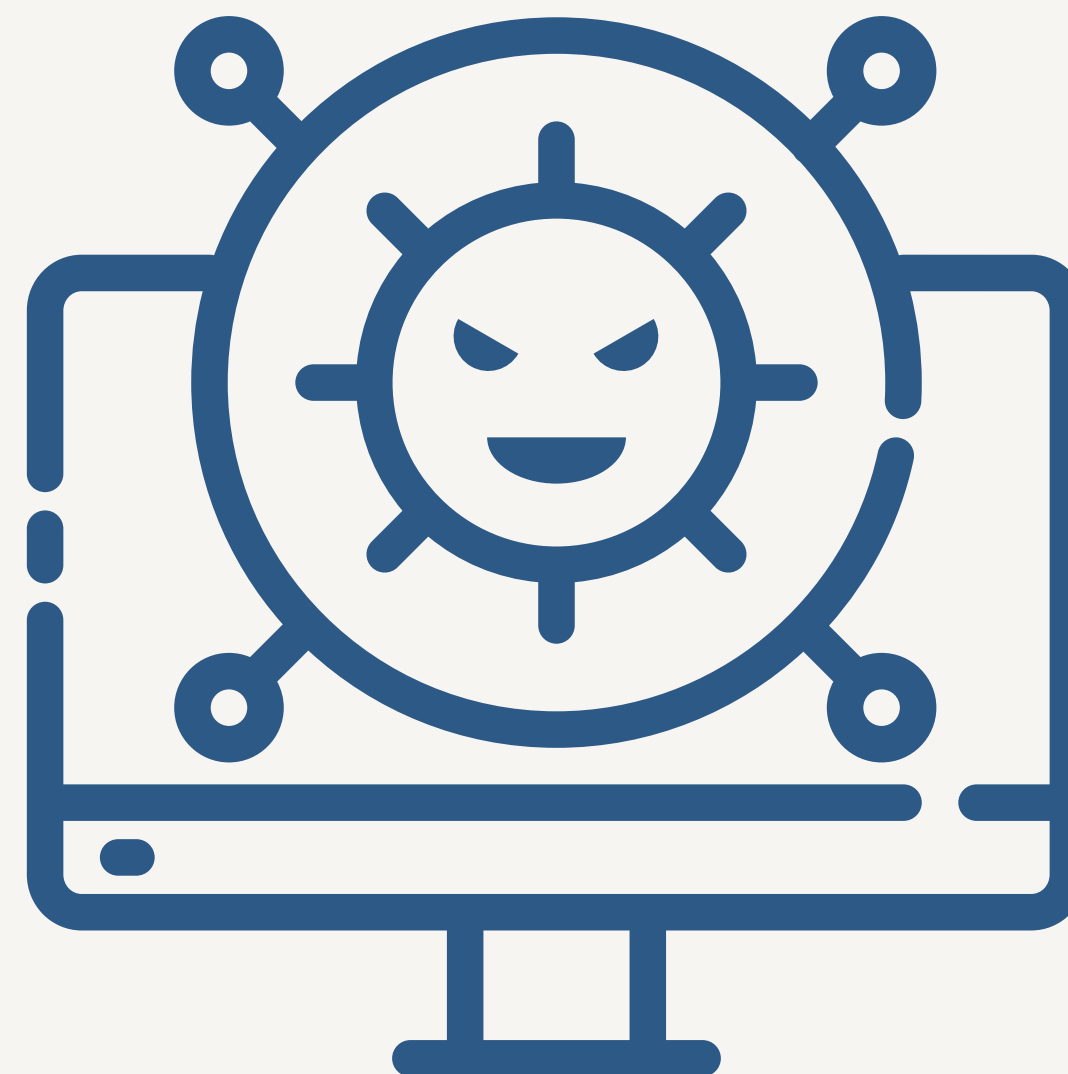
- 2 ЖЕРТВА ОТКРЫВАЕТ
ЗАРАЖЕННУЮ СТРАНИЦУ



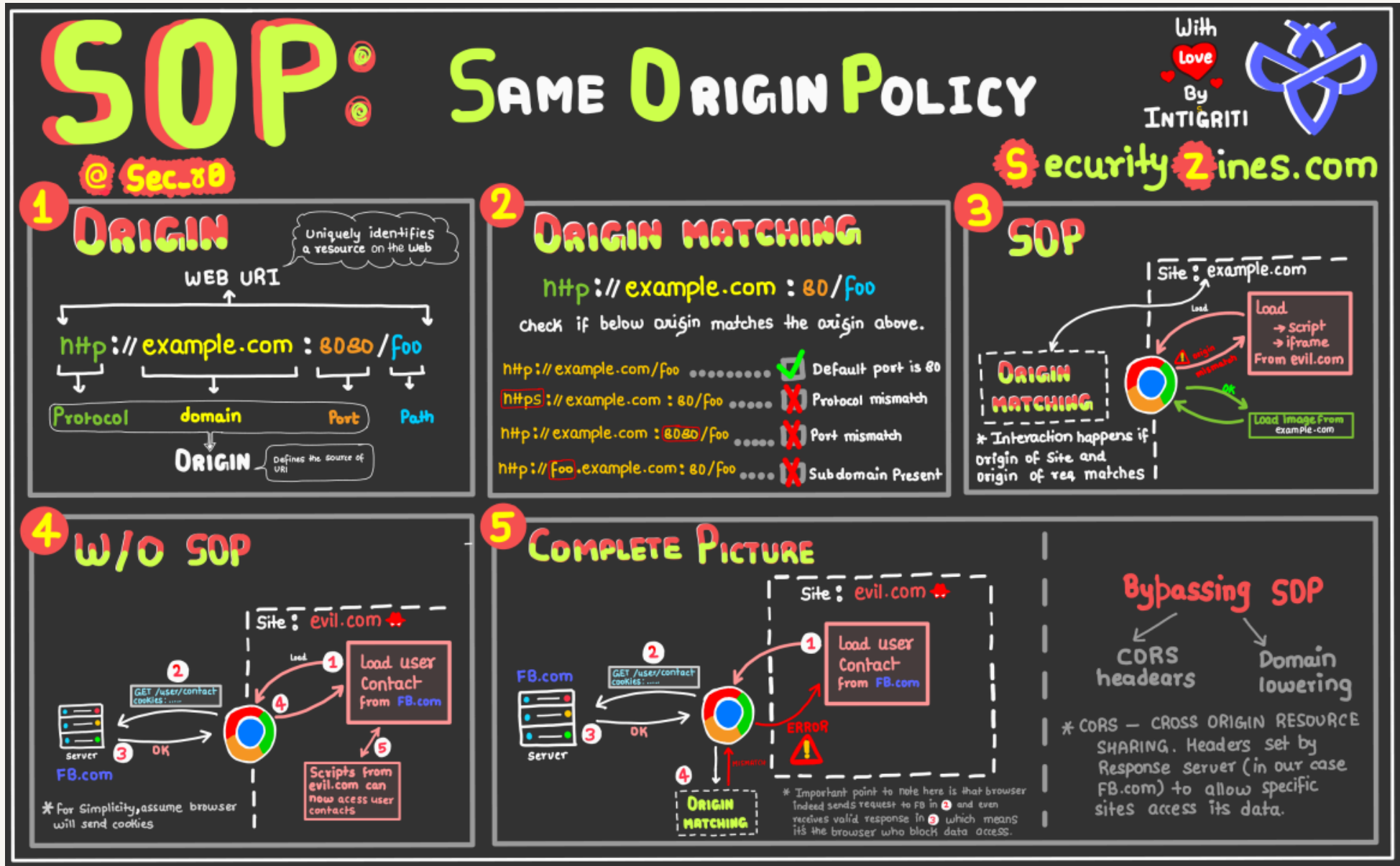
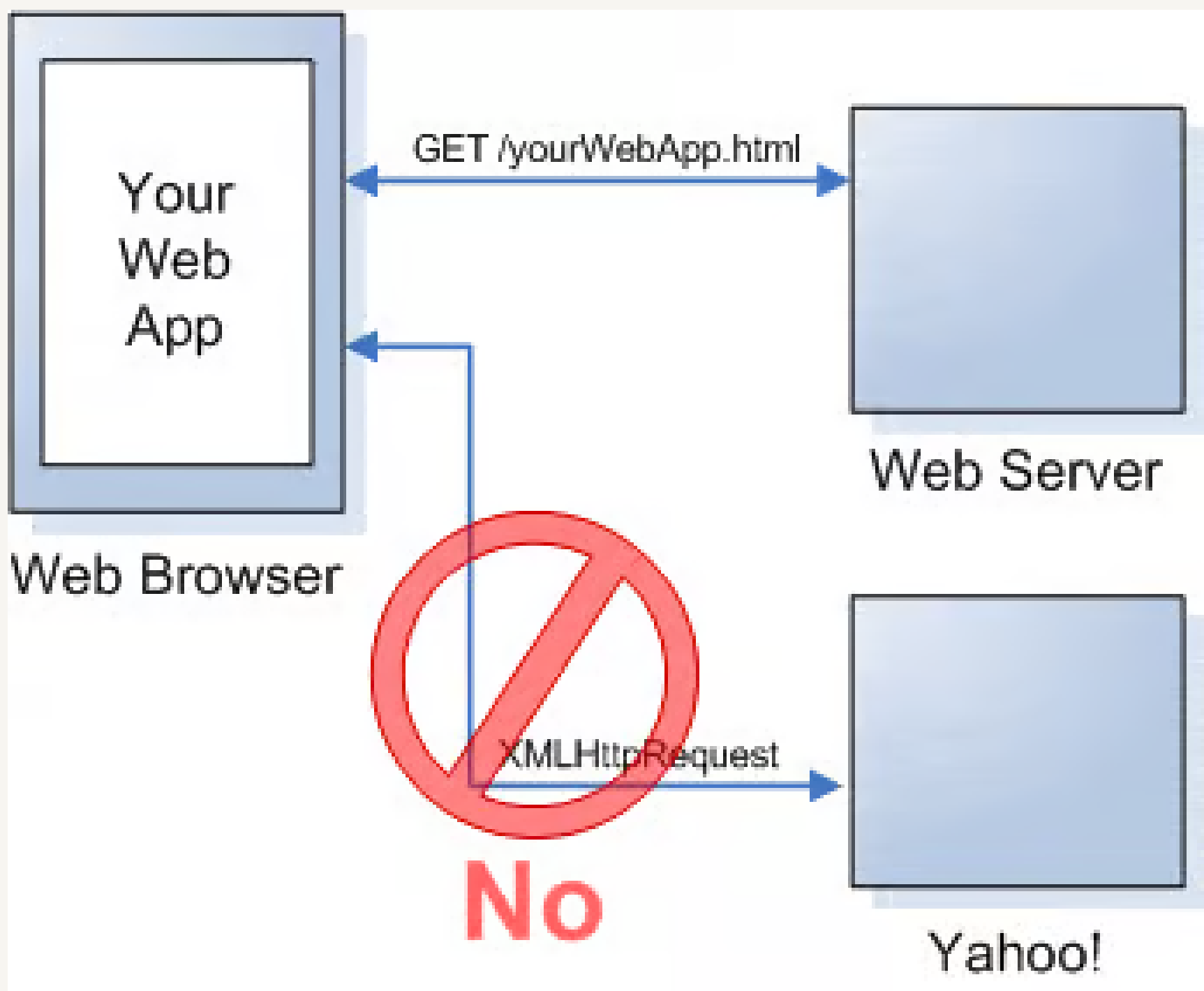
- 4 ТОКЕН ОТПРАВЛЯЕТСЯ ЗЛОУМЫШЛЕННИКУ



- 3 СКРИПТ ЧИТАЕТ CSRF ТОКЕН



Same-Origin Policy



CROSS-SITE HISTORY MANIPULATION (XSHM).

СПАСИБО ЗА ВНИМАНИЕ!

 +123-456-7890

 @reallygreatsite

 hello@reallygreatsite.com

CONTACT US 

